Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection)

Вариант	Целевой системный вызов (или tracepoint)	Условие аномального поведения (Фильтр ядра)
7	`sys_enter_mkdirat`	Попытка создания скрытой директории (имя начинается с точки `.`)

Подготовка стенда. Работа выполнена в Windows с использованием подсистемы WSL2 (Ubuntu 26.04, ядро 6.18.x-microsoft-standard-WSL2). Установлены пакеты bpfcc-tools и python3-bpfcc (BCC 0.35). Скрипты запускались с правами суперпользователя (sudo python3 ...). Docker Desktop для выполнения работы не использовался.

Задание 1. Базовый перехват запуска процессов

Разработан eBPF-зонд, прикреплённый к tracepoint syscalls:sys_enter_execve. Код ядра (C) при каждом запуске процесса получает PID (bpf_get_current_pid_tgid() >> 32), UID (bpf_get_current_uid_gid()), имя процесса comm (bpf_get_current_comm) и путь к запускаемому файлу (bpf_probe_read_user_str). Данные передаются в Python через буфер BPF_PERF_OUTPUT, где печатаются в виде таблицы.

```
#!/usr/bin/env python3

from bcc import BPF

prog = r"""

#include <linux/sched.h>

struct data_t {

    u32 pid;

    u32 uid;

    char comm[TASK_COMM_LEN];

    char fname[256];

};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_execve)

{

    struct data_t data = {};

    data.pid = bpf_get_current_pid_tgid() >> 32;

    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;

    bpf_get_current_comm(&data.comm, sizeof(data.comm));

    bpf_probe_read_user_str(&data.fname,     sizeof(data.fname),
args->filename);

    events.perf_submit(args, &data, sizeof(data));

    return 0;

}

"""

b = BPF(text=prog)

def print_event(cpu, data, size):

    e = b["events"].event(data)

    comm = e.comm.decode(errors="replace")

    fname = e.fname.decode(errors="replace")

    print(f"{e.pid:<8}{e.uid:<8}{comm:<18}{fname}")

b["events"].open_perf_buffer(print_event)

print(f"{'PID':<8}{'UID':<8}{'COMM':<18}{'FILE'}")

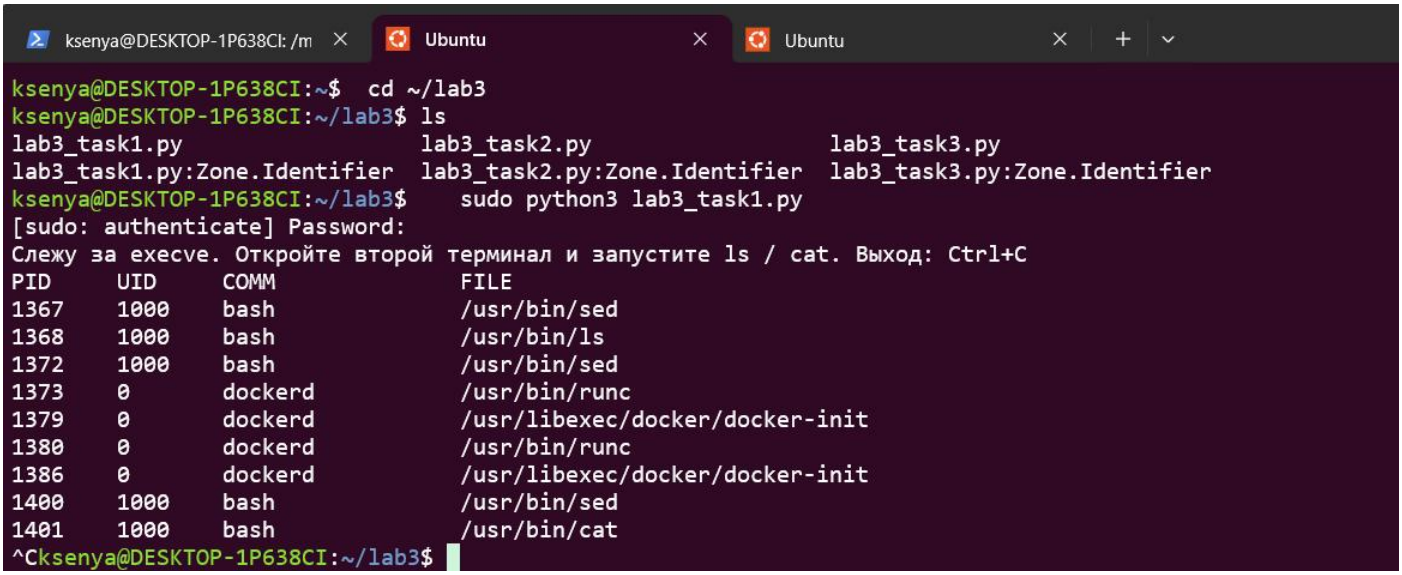
while True:

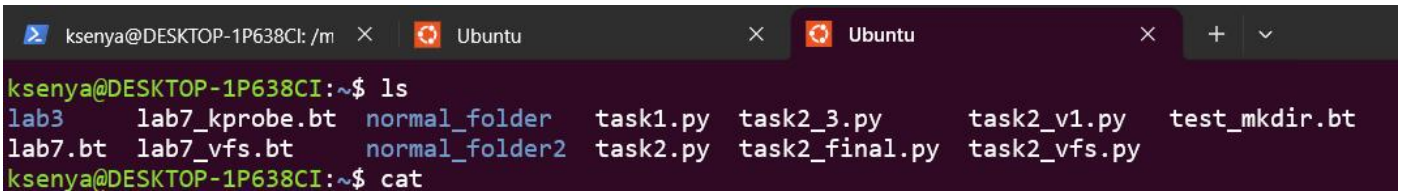
    try:

        b.perf_buffer_poll()

    except KeyboardInterrupt:

        break
```





В поле comm отображается bash, потому что на входе в execve процесс ещё носит имя родителя; имя исполняемого файла берётся из аргумента вызова.

Задание 2. Динамический анализ аномального поведения (вариант 7)

Условие: перехватывать sys_enter_mkdirat и сообщать только о попытках создания директории, имя которой начинается с точки (.).

Зонд копирует аргумент pathname из пользовательской памяти, определяет последний компонент пути (поиск / с конца строки) и проверяет, что он начинается с . . Имена . и .. исключены, так как это не скрытые директории. Фильтрация выполняется **целиком в ядре**: если имя не начинается с точки, обработчик завершается через return 0, и событие не покидает ядро. Функция alerts.perf_submit() вызывается только при совпадении условия, поэтому в пространство пользователя попадают исключительно аномальные события.

```
#!/usr/bin/env python3

from bcc import BPF

prog = r"""

#include <linux/sched.h>



#define PATH_LEN 256



#define IDX(x) ({ unsigned int _v = (x); asm volatile("" : "+r"(_v)); _v & 0xFF; })



struct alert_t {

    u32 pid;

    u32 uid;

    u32 mode;

    char comm[TASK_COMM_LEN];

    char path[PATH_LEN];

};

// Канал ядро -> Python. Сюда попадают ТОЛЬКО аномалии.

BPF_PERF_OUTPUT(alerts);



// Буфер-черновик: стек eBPF всего 512 байт, поэтому большую структуру

// держим в per-CPU массиве (по одному элементу на каждое ядро процессора)

BPF_PERCPU_ARRAY(scratch, struct alert_t, 1);



TRACEPOINT_PROBE(syscalls, sys_enter_mkdirat)

{

    int zero = 0;
```

```
    struct alert_t *a = scratch.lookup(&zero);

    if (!a)

        return 0;



    // Копируем путь (аргумент pathname) из памяти пользователя ОДИН раз.

    // n - длина строки вместе с завершающим '\0'

    long n = bpf_probe_read_user_str(a->path, sizeof(a->path), args->pathname);

    if (n <= 1 || n > PATH_LEN)        // пустая строка или ошибка чтения

        return 0;



    // --- Начало фильтрации в ядре ---

    // Находим начало ПОСЛЕДНЕГО компонента пути:

    //      "/tmp/.secret" -> ".secret",    ".hidden/" -> ".hidden"

    int end = n - 1;                              // индекс завершающего '\0'

    if (end > 1 && a->path[IDX(end - 1)] == '/')

        end--;                                // игнорируем один хвостовой '/'



    // Идём по строке С КОНЦА к началу до первого '/'.

    int start = 0;

    for (int i = 1; i < PATH_LEN; i++) {

        int idx = end - i;

        if (idx < 0)

            break;                       // '/' не нашли: весь путь - одно имя

        if (a->path[IDX(idx)] == '/') {

            start = idx + 1;

            break;

        }

    }

    // Имя не начинается с точки - легитимный вызов, отбрасываем ПРЯМО В ЯДРЕ

    if (a->path[IDX(start)] != '.')

        return 0;



    char c1 = a->path[IDX(start + 1)];

    char c2 = a->path[IDX(start + 2)];

    // "." (текущая директория) - не скрытая директория

    if (c1 == '\0' || c1 == '/')

        return 0;

    // ".." (родительская директория) - тоже не считаем аномалией

    if (c1 == '.' && (c2 == '\0' || c2 == '/'))

        return 0;

    // --- Конец фильтрации: дошли сюда - это аномалия ---



    a->pid  = bpf_get_current_pid_tgid() >> 32;

    a->uid  = bpf_get_current_uid_gid() & 0xFFFFFFFF;

    a->mode = args->mode;

    bpf_get_current_comm(&a->comm, sizeof(a->comm));



    // Отправка события в Python - только для аномалий

    alerts.perf_submit(args, a, sizeof(*a));

    return 0;

}

"""



b = BPF(text=prog)

def handle_alert(cpu, data, size):

    e = b["alerts"].event(data)

    comm = e.comm.decode(errors="replace")

    path = e.path.decode(errors="replace")

    print(f"[ALERT] Создание скрытой директории: PID={e.pid} UID={e.uid} "

          f"COMM={comm} PATH={path} MODE={oct(e.mode)}")

b["alerts"].open_perf_buffer(handle_alert)



print("Монитор mkdirat запущен (ищу скрытые директории). Выход: Ctrl+C")



while True:

    try:

        b.perf_buffer_poll()

    except KeyboardInterrupt:

        break
```

```
ksenya@DESKTOP-1P638CI:~/lab3$ sudo python3 lab3_task2.py
Монитор mkdirat запущен (ищу скрытые директории). Выход: Ctrl+C
[ALERT] Создание скрытой директории: PID=1662 UID=1000 COMM=python3 PATH=.secret_dir MODE=0o777
[ALERT] Создание скрытой директории: PID=1672 UID=1000 COMM=python3 PATH=/tmp/.hidden_test2 MODE=0o777
[ALERT] Создание скрытой директории: PID=1707 UID=1000 COMM=python3 PATH=.secret_dir4 MODE=0o777
[ALERT] Создание скрытой директории: PID=1717 UID=1000 COMM=python3 PATH=/tmp/.hidden_test4 MODE=0o777
```

```
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" norma
l_dir4
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" .secr
et_dir4
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" /tmp/
.hidden_test4
ksenya@DESKTOP-1P638CI:~/lab3$
```

Примечание: утилита mkdir в данной системе вызывает системный вызов mkdir (это подтверждено командой strace -e trace=mkdir,mkdirat), а не mkdirat. Поэтому событие mkdirat генерировалось вызовом os.mkdir(path, dir_fd=...) из Python.

Задание 3. Интеграция с пространством пользователя и аудит

Python-скрипт расширен: при получении алерта обработчик формирует строку с временной меткой (datetime.now()), PID, UID, именем процесса и описанием события и дописывает её в файл security_audit.log (функция write_audit, открытие файла в режиме "a").

Содержимое security_audit.log:

```
#!/usr/bin/env python3

# Лабораторная работа №3. Задание 3, вариант 7.

# То же, что в задании 2, но алерты дополнительно пишутся в security_audit.log

# Запуск: sudo python3 lab3_task3.py

import datetime

from bcc import BPF



LOG_FILE = "security_audit.log"     # создастся в текущей папке

prog = r"""

#include <linux/sched.h>



#define PATH_LEN 256

// Индекс в массиве path должен быть строго в пределах 0..255 (требование верификатора).

#define IDX(x) ({ unsigned int _v = (x); asm volatile("" : "+r"(_v)); _v & 0xFF; })



struct alert_t {

    u32 pid;

    u32 uid;

    u32 mode;

    char comm[TASK_COMM_LEN];

    char path[PATH_LEN];

};



BPF_PERF_OUTPUT(alerts);

BPF_PERCPU_ARRAY(scratch, struct alert_t, 1);



TRACEPOINT_PROBE(syscalls, sys_enter_mkdirat)

{

    int zero = 0;

    struct alert_t *a = scratch.lookup(&zero);

    if (!a)

        return 0;

    long n = bpf_probe_read_user_str(a->path, sizeof(a->path), args->pathname);

    if (n <= 1 || n > PATH_LEN)

        return 0;



    // --- Начало фильтрации в ядре: имя последнего компонента пути начинается с '.' ---

    int end = n - 1;                            // индекс завершающего '\0'

    if (end > 1 && a->path[IDX(end - 1)] == '/')

        end--;                                  // игнорируем один хвостовой '/'



    int start = 0;
```

```
    for (int i = 1; i < PATH_LEN; i++) {
        int idx = end - i;
        if (idx < 0)
            break;
        if (a->path[IDX(idx)] == '/') {
            start = idx + 1;
            break;
        }
    }
    if (a->path[IDX(start)] != '.')
        return 0;
    char c1 = a->path[IDX(start + 1)];
    char c2 = a->path[IDX(start + 2)];

    if (c1 == '\0' || c1 == '/')
        return 0;
    if (c1 == '.' && (c2 == '\0' || c2 == '/'))
        return 0;
    // --- Конец фильтрации ---
    a->pid  = bpf_get_current_pid_tgid() >> 32;
    a->uid  = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    a->mode = args->mode;
    bpf_get_current_comm(&a->comm, sizeof(a->comm));

    alerts.perf_submit(args, a, sizeof(*a));
    return 0;
}
"""
b = BPF(text=prog)
def write_audit(line):
    """Дописывает строку в конец файла аудита."""
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(line + "\n")

def handle_alert(cpu, data, size):
    e = b["alerts"].event(data)
    comm = e.comm.decode(errors="replace")
    path = e.path.decode(errors="replace")
    ts   =  datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    line = (f"{ts} | ALERT | PID={e.pid} | UID={e.uid} | COMM={comm} | "
            f"mkdirat: попытка создания скрытой директории PATH={path} MODE={oct(e.mode)}")

    print(line)          # на экран
    write_audit(line)    # и в файл security_audit.log
b["alerts"].open_perf_buffer(handle_alert)

print(f"Монитор запущен. Алерты пишутся в {LOG_FILE}. Выход: Ctrl+C")

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        break
```

```
^Cksenya@DESKTOP-1P638CI:~/lab3$ sudo python3 lab3_task3.py
Монитор запущен. Алерты пишутся в security_audit.log. Выход: Ctrl+C
2026-10-14 10:21:13 | ALERT | PID=1761 | UID=1000 | COMM=python3 | mkdirat: попытка создания скрытой директории PATH=.audit_test MODE=0o777
2026-10-14 10:21:20 | ALERT | PID=1771 | UID=1000 | COMM=python3 | mkdirat: попытка создания скрытой директории PATH=/tmp/.audit_hidden MODE=0o777
^Cksenya@DESKTOP-1P638CI:~/lab3$ cat security_audit.log
2026-10-14 10:21:13 | ALERT | PID=1761 | UID=1000 | COMM=python3 | mkdirat: попытка создания скрытой директории PATH=.audit_test MODE=0o777
2026-10-14 10:21:20 | ALERT | PID=1771 | UID=1000 | COMM=python3 | mkdirat: попытка создания скрытой директории PATH=/tmp/.audit_hidden MODE=0o777
ksenya@DESKTOP-1P638CI:~/lab3$
```

```
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" normal_dir5
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" .audit_test
ksenya@DESKTOP-1P638CI:~/lab3$ python3 -c "import os,sys; os.mkdir(sys.argv[1], dir_fd=os.open('.', os.O_RDONLY))" /tmp/.audit_hidden
ksenya@DESKTOP-1P638CI:~/lab3$
```

В записях присутствуют временная метка, PID и UID нарушителя и суть события.

Задание 4. Проектирование профилей защиты

1. Seccomp-BPF и eBPF kprobes. Seccomp-BPF — механизм ограничения: фильтр привязывается к процессу, наследуется потомками, не может быть снят и выполняется на входе в системный вызов. Он возвращает вердикт (ALLOW, ERRNO, KILL и др.), который ядро обязано исполнить, поэтому блокировка гарантирована. Ограничение: фильтр видит только номер вызова и значения аргументов-регистров и не разыменовывает указатели, поэтому не умеет фильтровать по строке пути.
eBPF kprobes/tracepoints — механизм трассировки: программа подключается глобально, может читать аргументы и память, но не возвращает вердикт: выполнение функции продолжается независимо от результата. Прерывание вызова из kprobe небезопасно: обработчик работает в атомарном контексте посреди кода ядра, и принудительный обрыв функции может оставить захваченные блокировки и счётчики ссылок, что приводит к утечкам, взаимоблокировкам и панике ядра. Подмена результата через bpf_override_return возможна лишь для функций, помеченных ALLOW_ERROR_INJECTION, и требует специальной конфигурации ядра. Сигнал bpf_send_signal асинхронен и может прийти после завершения вызова, а tracepoint вообще не влияет на выполнение. Кроме того, события могут теряться при переполнении буфера. Поэтому eBPF-зонды применяют для мониторинга и аудита, а для блокировки используют Seccomp, BPF LSM, SELinux/AppArmor.

2. TOCTOU. Аргумент pathname — указатель на строку в памяти пользователя. Tracepoint срабатывает до того, как ядро скопировало строку, поэтому строка читается дважды независимо: зондом (проверка) и ядром (использование). В многопоточном процессе другой поток может между этими моментами изменить строку: зонд увидит безобидное имя, а ядро создаст скрытую директорию, и мониторинг будет обойдён. Кроме того, bpf_probe_read_user_str не может подгрузить выгруженную страницу и возвращает ошибку.
Меры противодействия: копировать строку один раз в локальный буфер функцией bpf_probe_read_user_str и работать только с копией; проверять данные после того, как ядро само скопировало их (kprobe на do_mkdirat/filename_create, где struct filename уже в памяти ядра, либо BPF LSM-хук security_path_mkdir); дополнительно использовать sys_exit_mkdirat с корреляцией по pid_tgid через хеш-карту; считать ошибки чтения подозрительным событием.

Вывод: в ходе работы освоена разработка eBPF-программ с использованием BCC: реализованы перехват execve и мониторинг скрытых директорий по mkdirat с фильтрацией на стороне ядра и записью аудита в файл. Приобретены навыки анализа ошибок верификатора ядра (ограничения на размер состояния программы и проверка границ индексов), отладки с помощью strace и работы в среде WSL2. Ограничения технологии: необходимость прав root и привязка к версии ядра; жёсткие требования верификатора к коду; невозможность надёжной блокировки из kprobe/tracepoint; потери событий при перегрузке буфера; уязвимость к гонкам TOCTOU при чтении аргументов-указателей до их копирования ядром.